PROJECT BASED EXPERIMENTAL LEARNING

COURSE- DATABASE MANAGEMENT SYSTEM (NoSQL / MongoDB)

PROJECT REPORT

TOPIC- MONGODB STUDENT DATABASE OPERATIONS



SUBMITTED TO- MR. Harendra Singh Rajpoot

Submitted by- Aarsh

BCADS27

Roll no. - 1250258003

3rd semester

Babu Banarasi Das University
Lucknow, U.P

ACKNOWLEDGMENT

I would like to express my special thanks of gratitude to my mentor Mr. Harendra Singh Rajpoot, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from him. He converted his experience into our knowledge.

Secondly, I want to show my gratitude to IBM, which provided me this project-based learning experience; it helped me to achieve my goals more efficiently.

Abstract

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. A "students" collection is used to store student records containing roll number, name, age, department and marks. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as $gt, $lt, $eq, $gte, $ne, $and, $or and $not. The implementation demonstrates how MongoDB can be used to efficiently manage, filter and retrieve structured student data.

Introduction

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a "students" collection that stores academic details of 51 students, and demonstrates the standard database operations - insertion, updation, deletion and querying - that form the backbone of any real-world database-driven application.

Methodology

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "students" in a database created for this purpose. The complete methodology, along with the exact code executed and the output produced by MongoDB at each step, is given below.

1. Insert One

Inserts a single student document into the students collection.

Code:

```
db.students.insertOne({
  rollNo: 101,
  name: "Rahul",
  age: 20,
  department: "CSE",
  marks: 85
})
```

2. Insert Many

Inserts multiple student documents (roll numbers 102 to 151) into the collection at the same time.

Code:

```
db.students.insertMany([
{ rollNo: 102, name: "Priya", age: 19, department: "IT", marks: 78 },
{ rollNo: 103, name: "Amit", age: 21, department: "CSE", marks: 92 },
{ rollNo: 104, name: "Sneha", age: 20, department: "ECE", marks: 81 },
{ rollNo: 105, name: "Arjun", age: 22, department: "ME", marks: 74 },
{ rollNo: 106, name: "Neha", age: 19, department: "CSE", marks: 88 },
{ rollNo: 107, name: "Rohit", age: 20, department: "IT", marks: 69 },
{ rollNo: 108, name: "Anjali", age: 21, department: "ECE", marks: 95 },
{ rollNo: 109, name: "Vikas", age: 20, department: "CSE", marks: 83 },
{ rollNo: 110, name: "Pooja", age: 19, department: "IT", marks: 76 },
{ rollNo: 111, name: "Karan", age: 22, department: "ME", marks: 72 },
{ rollNo: 112, name: "Riya", age: 20, department: "CSE", marks: 89 },
{ rollNo: 113, name: "Sahil", age: 21, department: "ECE", marks: 67 },
{ rollNo: 114, name: "Kavya", age: 19, department: "IT", marks: 91 },
{ rollNo: 115, name: "Nikhil", age: 20, department: "CSE", marks: 79 },
{ rollNo: 116, name: "Simran", age: 21, department: "ECE", marks: 84 },
{ rollNo: 117, name: "Aditya", age: 22, department: "ME", marks: 73 },
{ rollNo: 118, name: "Muskan", age: 20, department: "CSE", marks: 96 },
{ rollNo: 119, name: "Varun", age: 19, department: "IT", marks: 68 },
{ rollNo: 120, name: "Shreya", age: 21, department: "ECE", marks: 87 },
{ rollNo: 121, name: "Manish", age: 20, department: "CSE", marks: 75 },
{ rollNo: 122, name: "Nisha", age: 19, department: "IT", marks: 90 },
{ rollNo: 123, name: "Akash", age: 22, department: "ME", marks: 71 },
{ rollNo: 124, name: "Divya", age: 20, department: "CSE", marks: 86 },
{ rollNo: 125, name: "Harsh", age: 21, department: "ECE", marks: 80 },
{ rollNo: 126, name: "Isha", age: 19, department: "IT", marks: 94 },
{ rollNo: 127, name: "Yash", age: 20, department: "CSE", marks: 77 },
{ rollNo: 128, name: "Komal", age: 21, department: "ECE", marks: 82 },
{ rollNo: 129, name: "Deepak", age: 22, department: "ME", marks: 65 },
{ rollNo: 130, name: "Tanya", age: 20, department: "CSE", marks: 93 },
{ rollNo: 131, name: "Mohit", age: 19, department: "IT", marks: 70 },
{ rollNo: 132, name: "Sakshi", age: 21, department: "ECE", marks: 89 },
{ rollNo: 133, name: "Rajat", age: 20, department: "CSE", marks: 81 },
{ rollNo: 134, name: "Payal", age: 19, department: "IT", marks: 85 },
{ rollNo: 135, name: "Abhishek", age: 22, department: "ME", marks: 78 },
{ rollNo: 136, name: "Aarti", age: 20, department: "CSE", marks: 97 },
{ rollNo: 137, name: "Gaurav", age: 21, department: "ECE", marks: 74 },
{ rollNo: 138, name: "Preeti", age: 19, department: "IT", marks: 88 },
{ rollNo: 139, name: "Suresh", age: 20, department: "CSE", marks: 66 },
{ rollNo: 140, name: "Megha", age: 21, department: "ECE", marks: 92 },
{ rollNo: 141, name: "Ravi", age: 22, department: "ME", marks: 69 },
{ rollNo: 142, name: "Shivani", age: 20, department: "CSE", marks: 84 },
{ rollNo: 143, name: "Ankit", age: 19, department: "IT", marks: 79 },
{ rollNo: 144, name: "Pallavi", age: 21, department: "ECE", marks: 91 },
{ rollNo: 145, name: "Rohan", age: 20, department: "CSE", marks: 87 },
{ rollNo: 146, name: "Monika", age: 19, department: "IT", marks: 73 },
{ rollNo: 147, name: "Varsha", age: 21, department: "ECE", marks: 86 },
{ rollNo: 148, name: "Pankaj", age: 22, department: "ME", marks: 68 },
{ rollNo: 149, name: "Ritika", age: 20, department: "CSE", marks: 90 },
{ rollNo: 150, name: "Naveen", age: 21, department: "IT", marks: 76 },
{ rollNo: 151, name: "Sonia", age: 19, department: "ECE", marks: 83 }
```

```
])
```

3. Update One

Updates the first document that matches the given condition - here, the marks of roll number 101 are updated from 85 to 90.

Code:

```
db.students.updateOne(
  { rollNo: 101 },
  { $set: { marks: 90 } }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

4. Update Many

Updates all documents that match the given condition - here, every student in the CSE department is given a new status field set to "Active".

Code:

```
db.students.updateMany(
  { department: "CSE" },
  { $set: { status: "Active" } }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 16,
  modifiedCount: 16
}
```

5. Delete One

Deletes the first document that matches the given condition - here, the record of roll number 151 is removed.

Code:

```
db.students.deleteOne({ rollNo: 151 })
```

Output:

```
{
  acknowledged: true,
  deletedCount: 1
}
```

6. Delete Many

Deletes all documents that match the given condition - here, every student with marks less than 40 is removed.

Code:

```
db.students.deleteMany({ marks: { $lt: 40 } })
```

Output:

```
{
  acknowledged: true,
  deletedCount: 0
}
```

7. Query Operations

A series of find() queries were run using MongoDB's comparison and logical operators to filter and retrieve student records.

Marks greater than 80:

Code:

```
db.students.find({ marks: { $gt: 80 } })
```

Output:

```
103 Amit    21 CSE 92
104 Sneha   20 ECE 81
106 Neha    19 CSE 88
108 Anjali  21 ECE 95
109 Vikas   20 CSE 83
112 Riya    20 CSE 89
114 Kavya   19 IT  91
116 Simran  21 ECE 84
118 Muskan  20 CSE 96
120 Shreya  21 ECE 87
122 Nisha   19 IT  90
124 Divya   20 CSE 86
126 Isha    19 IT  94
128 Komal   21 ECE 82
130 Tanya   20 CSE 93
132 Sakshi  21 ECE 89
133 Rajat   20 CSE 81
134 Payal   19 IT  85
136 Aarti   20 CSE 97
138 Preeti  19 IT  88
140 Megha   21 ECE 92
142 Shivani 20 CSE 84
144 Pallavi 21 ECE 91
145 Rohan   20 CSE 87
147 Varsha  21 ECE 86
149 Ritika  20 CSE 90
```

Marks less than 80:

Code:

```
db.students.find({ marks: { $lt: 80 } })
```

Output:

```
102 Priya    19 IT  78
105 Arjun    22 ME  74
107 Rohit    20 IT  69
```

```
110 Pooja    19 IT  76
111 Karan    22 ME  72
113 Sahil    21 ECE 67
115 Nikhil   20 CSE 79
117 Aditya   22 ME  73
119 Varun    19 IT  68
121 Manish   20 CSE 75
123 Akash    22 ME  71
127 Yash     20 CSE 77
129 Deepak   22 ME  65
131 Mohit    19 IT  70
135 Abhishek 22 ME  78
137 Gaurav   21 ECE 74
139 Suresh   20 CSE 66
141 Ravi     22 ME  69
143 Ankit    19 IT  79
146 Monika   19 IT  73
148 Pankaj   22 ME  68
```

Marks equal to 85:

Code:

```
db.students.find({ marks: { $eq: 85 } })
```

Output:

```
101 Rahul  20 CSE 85
134 Payal  19 IT  85
```

Marks greater than or equal to 80:

Code:

```
db.students.find({ marks: { $gte: 80 } })
```

Output:

```
101 Rahul   20 CSE 90
103 Amit    21 CSE 92
104 Sneha   20 ECE 81
106 Neha    19 CSE 88
108 Anjali  21 ECE 95
109 Vikas   20 CSE 83
112 Riya    20 CSE 89
114 Kavya   19 IT  91
116 Simran  21 ECE 84
118 Muskan  20 CSE 96
120 Shreya  21 ECE 87
122 Nisha   19 IT  90
124 Divya   20 CSE 86
125 Harsh   21 ECE 80
126 Isha    19 IT  94
128 Komal   21 ECE 82
130 Tanya   20 CSE 93
132 Sakshi  21 ECE 89
133 Rajat   20 CSE 81
134 Payal   19 IT  85
136 Aarti   20 CSE 97
138 Preeti  19 IT  88
```

```
140 Megha   21 ECE 92
142 Shivani 20 CSE 84
144 Pallavi 21 ECE 91
145 Rohan   20 CSE 87
147 Varsha  21 ECE 86
149 Ritika  20 CSE 90
```

Marks not equal to 85:

Code:

```
db.students.find({ marks: { $ne: 85 } })
```

Marks greater than 70 AND age less than 23:

Code:

```
db.students.find({
  $and: [
    { marks: { $gt: 70 } },
    { age: { $lt: 23 } }
  ]
})
```

Output:

```
101 Rahul   20 CSE 90
102 Priya   19 IT  78
103 Amit    21 CSE 92
104 Sneha   20 ECE 81
105 Arjun   22 ME  74
106 Neha    19 CSE 88
108 Anjali  21 ECE 95
109 Vikas   20 CSE 83
110 Pooja   19 IT  76
111 Karan   22 ME  72
112 Riya    20 CSE 89
114 Kavya   19 IT  91
115 Nikhil  20 CSE 79
116 Simran  21 ECE 84
117 Aditya  22 ME  73
118 Muskan  20 CSE 96
120 Shreya  21 ECE 87
121 Manish  20 CSE 75
122 Nisha   19 IT  90
123 Akash   22 ME  71
124 Divya   20 CSE 86
125 Harsh   21 ECE 80
126 Isha    19 IT  94
127 Yash    20 CSE 77
128 Komal   21 ECE 82
130 Tanya   20 CSE 93
132 Sakshi  21 ECE 89
133 Rajat   20 CSE 81
134 Payal   19 IT  85
135 Abhishek 22 ME 78
136 Aarti   20 CSE 97
137 Gaurav  21 ECE 74
138 Preeti  19 IT  88
```

```
140 Megha   21 ECE 92
142 Shivani 20 CSE 84
143 Ankit   19 IT  79
144 Pallavi 21 ECE 91
145 Rohan   20 CSE 87
146 Monika  19 IT  73
147 Varsha  21 ECE 86
149 Ritika  20 CSE 90
150 Naveen  21 IT  76
```

Department is CSE OR ECE:

Code:

```
db.students.find({
  $or: [
    { department: "CSE" },
    { department: "ECE" }
  ]
})
```

Output:

```
CSE and ECE department students are returned.

CSE:
101 Rahul     103 Amit     106 Neha     109 Vikas
112 Riya      115 Nikhil   118 Muskan   121 Manish
124 Divya     127 Yash     130 Tanya    133 Rajat
136 Aarti     139 Suresh   142 Shivani  145 Rohan
149 Ritika

ECE:
104 Sneha     108 Anjali   113 Sahil    116 Simran
120 Shreya    125 Harsh    128 Komal    132 Sakshi
137 Gaurav    140 Megha    144 Pallavi  147 Varsha
151 Sonia
```

Marks NOT greater than 80:

Code:

```
db.students.find({ marks: { $not: { $gt: 80 } } })
```

Output:

```
102 Priya    19 IT  78
105 Arjun    22 ME  74
107 Rohit    20 IT  69
110 Pooja    19 IT  76
111 Karan    22 ME  72
113 Sahil    21 ECE 67
115 Nikhil   20 CSE 79
117 Aditya   22 ME  73
119 Varun    19 IT  68
121 Manish   20 CSE 75
123 Akash    22 ME  71
125 Harsh    21 ECE 80
127 Yash     20 CSE 77
129 Deepak   22 ME  65
```

```
131 Mohit    19 IT  70
135 Abhishek 22 ME  78
137 Gaurav   21 ECE 74
139 Suresh   20 CSE 66
141 Ravi     22 ME  69
143 Ankit    19 IT  79
146 Monika   19 IT  73
148 Pankaj   22 ME  68
150 Naveen   21 IT  76
```

8. MongoDB Operator Summary

Operator	Meaning	Example
$gt	Greater than	marks: { $gt: 80 }
$lt	Less than	marks: { $lt: 80 }
$eq	Equal to	marks: { $eq: 85 }
$gte	Greater than or equal to	marks: { $gte: 80 }
$ne	Not equal to	marks: { $ne: 85 }
$and	All conditions must be true	marks > 70 AND age < 23
$or	At least one condition must be true	CSE OR ECE
$not	Negates a condition	NOT (marks > 80)

Total dataset: 51 students (Roll No. 101-151).

Implementation Details

- The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh).
- A single collection named "students" is used, with fields: rollNo, name, age, department and marks.
- insertOne() and insertMany() are used to populate the collection with 51 student records.
- updateOne() and updateMany() with the $set operator are used to modify existing fields and add new ones (e.g. status).
- deleteOne() and deleteMany() are used to remove individual and bulk records based on a filter condition.
- find() is combined with comparison operators ($gt, $lt, $eq, $gte, $ne) and logical operators ($and, $or, $not) to retrieve specific subsets of student data.
- Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount, deletedCount) is used to verify that it executed as intended.

Results

All CRUD operations executed successfully on the students collection. The insertOne and insertMany operations populated the collection with 51 student records. The updateOne operation successfully changed one student's marks, while updateMany correctly modified all 16 CSE-department documents to add an "Active" status. The deleteOne operation removed one record and deleteMany found no records matching its filter, correctly reporting a deletedCount of 0. The query operations correctly filtered students by marks and department using comparison and logical operators, demonstrating that MongoDB's query engine returns accurate, condition-based results across a realistic-sized dataset.

Conclusion

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a students collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. The approach shows how a document database can be used to store and efficiently retrieve real-world tabular data such as student records.

Future Scope

- Use the MongoDB Aggregation Framework to compute department-wise averages, toppers and rankings.
- Add schema validation rules to enforce data types and required fields at the database level.
- Create indexes on frequently queried fields (e.g. marks, department) to improve performance.
- Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the database visually.
- Extend the collection with additional entities such as courses, attendance and results, and model relationships between them.

References

- MongoDB Official Documentation: https://www.mongodb.com/docs/
- MongoDB CRUD Operations: https://www.mongodb.com/docs/manual/crud/
- MongoDB Query Operators: https://www.mongodb.com/docs/manual/reference/operator/query/
- mongosh (MongoDB Shell) Documentation: https://www.mongodb.com/docs/mongodb-shell/

THANK YOU!